



SAPIENZA
UNIVERSITÀ DI ROMA

Oltre il Gold Standard: analisi comparativa tra metriche tradizionali e approcci LLM-as-a-Judge per la valutazione del web parsing

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica
Ingegneria Informatica e Automatica

Ludovica Fiocchetta

Matricola 2103669

Relatore

Professore Roberto Navigli

Anno Accademico 2025/2026

**Oltre il Gold Standard: analisi comparativa tra metriche tradizionali e approcci
LLM-as-a-Judge per la valutazione del web parsing**

Laurea Triennale. Sapienza Università di Roma

© 2026 Ludovica Fiocchetta. Tutti i diritti riservati

Questa tesi è stata composta con $\text{\LaTeX}$ e la classe Sapthesis.

Email dell'autore: fiocchetta.2103669@studenti.uniroma1.it

Indice

1	Introduzione	1
1.1	Contesto generale	1
1.2	Motivazione: il progetto Chat Minerva	2
1.3	Obiettivi e contributo dell'elaborato	2
1.4	Struttura della tesi	4
2	Lavori correlati	5
2.1	Tecniche di estrazione	5
2.2	Metriche tradizionali	5
2.3	Metriche semantiche	5
2.4	LLM-as-a-Judge	5
3	Sistema presentato: architettura e implementazione	7
3.1	Panoramica dell'infrastruttura	7
3.2	Progettazione del backend	8
3.2.1	Gestione degli endpoint	9
3.2.2	Logica dei parser	11
3.2.3	Motore di calcolo delle metriche	13
3.3	Progettazione del frontend	15
3.3.1	Architettura API e gestione delle latenze	16
3.3.2	Accesso al sistema e interfaccia di analisi	18
3.3.3	Pannello CRUD e dashboard statistica	20
3.4	Gestione dei dati	21
3.4.1	Modello relazionale	21
3.4.2	Inizializzazione e popolamento	23
4	Paradigma LLM-as-a-Judge	25
4.1	Inquadramento metodologico	25
4.2	Integrazione del motore Ollama	26
4.3	Ingegneria del prompt e gestione del contesto	26
4.4	Determinismo del modello e validazione strutturale	28
5	Risultati ottenuti	31
5.1	Impostazione degli esperimenti	31
5.2	Analisi delle metriche tradizionali e avanzate	32
5.3	Valutazione LLM-as-a-Judge	34
5.4	Limiti del sistema	35

6 Conclusioni e sviluppi futuri	37
---------------------------------	----

Capitolo 1

Introduzione

1.1 Contesto generale

Nell'attuale contesto informativo tecnologico, il World Wide Web rappresenta il più vasto bacino di conoscenza non strutturata mai esistito. Con lo sviluppo dei Large Language Models, l'abilità di elaborare e comprendere il linguaggio naturale è diventata una necessità, poiché l'efficacia di questi modelli è fortemente dipendente dalla qualità dei dati forniti in input. Gli LLM hanno ancora una capacità limitata di manipolare con precisione la conoscenza, infatti falliscono nell'accedere a fonti precise e tendono ad avere allucinazioni su compiti *knowledge-intensive*. Le sfide più note includono:

- Presentazione di informazioni false quando non si ha la risposta;
- Presentazione di informazioni obsolete o generiche quando l'utente si aspetta una risposta specifica e attuale;
- Creazione di una risposta da fonti non autorevoli;
- Creazione di risposte imprecise a causa della confusione terminologica, in cui diverse fonti di addestramento utilizzano la stessa terminologia per parlare di argomenti diversi.

Per questi motivi il processo di estrazione di dati puliti assume un ruolo fondamentale. Nel paradigma della *Retrieval-Augmented Generation* (RAG) [1], architettura che ottimizza le prestazioni di un modello di intelligenza artificiale collegandolo a knowledge base esterne, il *web scraping* tradizionale, basato su regole deterministiche e selettori del Document Object Model (DOM), risulta spesso fragile di fronte alla grande diversità delle strutture HTML. Questo approccio spesso lascia insieme al testo utile anche "rumore", come menu di navigazione, banner pubblicitari e riferimenti ad altre pagine, difetto che può tradursi in allucinazioni nelle risposte degli LLM. Un approccio più robusto richiede sistemi di *web parsing* semantico, che consentono di migliorare i risultati essendo in grado di scansionare grandi database e di isolare esclusivamente le informazioni rilevanti nella pagina, ignorando tutti gli elementi di disturbo.

1.2 Motivazione: il progetto Chat Minerva

Questa necessità operativa si manifesta concretamente all'interno del progetto Chat Minerva sviluppato da Sapienza Università di Roma, per il quale è indispensabile fornire al modello testi in input totalmente puliti al fine di garantire risposte attendibili. Chat Minerva è un modello linguistico avanzato che si pone l'obiettivo di fornire un agente di chat affidabile, capace di rispondere in modo accurato e contestualizzato alle molteplici domande da parte degli utenti. Si basa su un'architettura *Retrieval-Augmented Generation* (RAG), così da fare riferimento a una base di conoscenza al di fuori dei suoi dati di addestramento originali prima di generare una risposta [2], il cui flusso operativo è illustrato in Figura 1.1.

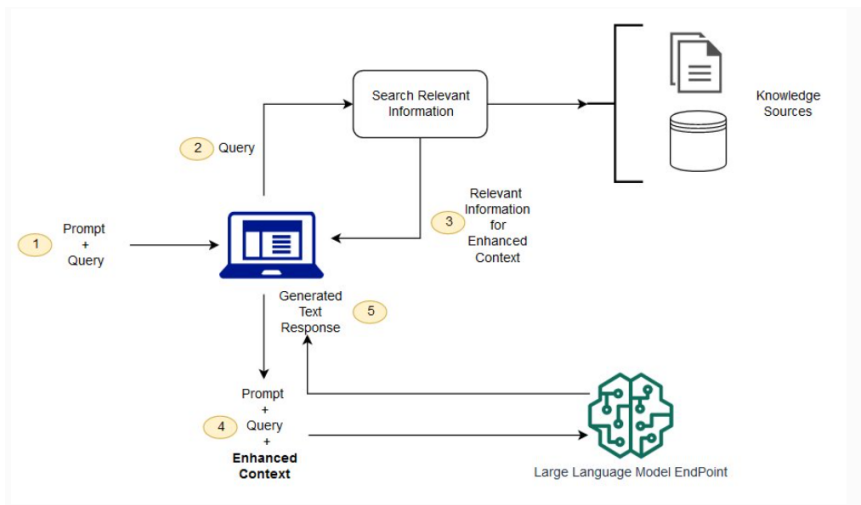


Figura 1.1. Rappresentazione di un'architettura RAG. Il framework sviluppato assicura la qualità dei dati estratti prima del loro inserimento nelle *Knowledge Sources* [2].

Come evidenziato dallo schema, quando l'utente formula una query (passaggi 1 e 2), il sistema ricerca informazioni pertinenti all'interno delle proprie fonti di conoscenza esterne per arricchire il contesto del prompt (passaggi 3 e 4). Se i parser che alimentano questi database estraggono testo sporco o frammenti di codice HTML, l'agente genererà risposte errate o allucinate (passaggio 5). Affinché il modello non generi informazioni sbagliate o non pertinenti è essenziale che i dati forniti in fase di inferenza siano estremamente puliti. Se i parser utilizzati per alimentare Minerva estraggono testo sporco o formattazioni residue, la qualità della risposta del modello degrada drasticamente. È nata quindi la necessità ingegneristica di sviluppare un'infrastruttura capace di testare, misurare e validare in modo oggettivo le performance di diversi sistemi di web parsing, garantendo che al Chatbot venga passata esclusivamente la conoscenza utile e strutturata di cui ha bisogno.

1.3 Obiettivi e contributo dell'elaborato

Il seguente elaborato ha dunque come scopo principale la descrizione di un sistema di estrazione automatizzata di testo pulito da quattro domini internet, e la succes-

siva analisi comparativa sulla valutazione della qualità dei parser. Questo studio metterà a confronto le metriche matematiche tradizionali, che basano il giudizio sulla comparazione con un *gold standard*, con l'innovativo paradigma *LLM-as-a-Judge*, esplorato anche in scenari *reference-free*. Nello specifico, il sistema si articola nei seguenti punti:

- **Sviluppo della pipeline di estrazione e valutazione:** implementazione di un'infrastruttura Python capace di estrarre testo pulito e misurare in modo oggettivo le performance dei sistemi di *web parsing*.
- **Analisi con metriche classiche e avanzate:** valutazione dei *parser* basata sul confronto dei testi con un *gold standard*, calcolando metriche rigorose quali Precision e Recall, ma anche semantiche come BERTScore e METEOR.
- **Sperimentazione del paradigma LLM-as-a-Judge:** introduzione di un approccio moderno in cui un Large Language Model agisce da giudice, esplorando l'efficacia di valutazioni *reference-free* per superare la rigidità delle metriche tradizionali.

Il contributo principale dell'elaborato risiede proprio in questo doppio livello della pipeline di valutazione, evidenziato in Figura 1.2. Infatti, da un lato si ha un giudizio basato su un *gold standard*, in cui il contenuto estratto viene confrontato con un testo di riferimento salvato all'interno di un database relazionale. In questa fase vengono calcolate metriche classiche basate sulla sovrapposizione insiemistica e sulle distanze operative (come la *Jaccard Similarity*), affiancate da metriche semantiche avanzate (come BERTScore e METEOR). Dall'altro lato, si esplora l'approccio *LLM-as-a-Judge*, chiamando un modello linguistico per valutare la coerenza e la pulizia del testo estratto anche in modalità *reference-free*. Questo permette di superare il limite intrinseco del *gold standard*, che richiede una costosa annotazione manuale preventiva. L'LLM agisce come giudice autonomo: attraverso tecniche di *prompt engineering* e pipeline difensive contro i colli di bottiglia computazionali, è in grado di analizzare la pulizia dell'estrazione anche su domini web sconosciuti, fornendo un punteggio quantitativo e un commento qualitativo pronto all'uso. Questo studio dimostrerà come l'integrazione di approcci semantici avanzati sia essenziale per certificare la qualità del testo estratto, garantendo l'affidabilità di sistemi complessi come Chat Minerva. A supporto di tale studio, l'intero framework software sviluppato è rilasciato in modalità *open-source* ed è liberamente consultabile su GitHub¹.

¹Repository ufficiale del progetto: <https://github.com/ludofiocchetta-glitch/tesi-parser-and-llm-as-a-judge>

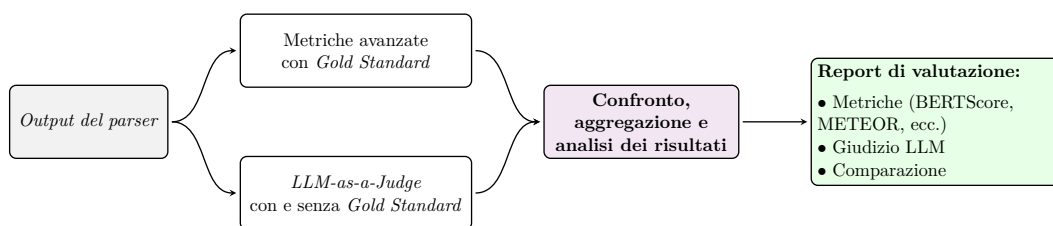


Figura 1.2. Panoramica architetturale e metodologica del sistema di valutazione comparativa implementato nel progetto.

1.4 Struttura della tesi

La trattazione è organizzata come segue:

- **Capitolo 2:** analizza i lavori correlati, descrivendo l'evoluzione delle tecniche di *web parsing* e presentando il fondamento teorico delle metriche di valutazione impiegate nel progetto, dagli approcci tradizionali e semantici fino all'introduzione del paradigma *LLM-as-a-Judge*.
- **Capitolo 3:** descrive in dettaglio la progettazione architetturale del sistema, illustrando dettagliatamente tutte le scelte informatiche adottate.
- **Capitolo 4:** esplora l'implementazione degli approcci *LLM-as-a-Judge*, analizzando la costruzione dei *prompt* di valutazione e l'integrazione con i modelli in esecuzione locale.
- **Capitolo 5:** presenta l'impostazione degli esperimenti sui domini selezionati e discute criticamente i risultati ottenuti, confrontando i punteggi delle metriche deterministiche con il giudizio qualitativo dell'intelligenza artificiale.
- **Capitolo 6:** riassume i traguardi raggiunti e delinea i possibili sviluppi futuri per il potenziamento dell'ecosistema.

Capitolo 2

Lavori correlati

2.1 Tecniche di estrazione

2.2 Metriche tradizionali

2.3 Metriche semantiche

2.4 LLM-as-a-Judge

Capitolo 3

Sistema presentato: architettura e implementazione

3.1 Panoramica dell'infrastruttura

Questo capitolo descrive nel dettaglio l'architettura ingegneristica alla base del framework di valutazione. Il sistema ha come obiettivo principale l'estrazione automatizzata di testo pulito dalle pagine di quattro domini internet tramite la libreria *Crawl4AI* e la successiva valutazione dell'accuratezza dei parser, mediante il confronto tra il testo estratto dal *crawler* e il *gold standard* di riferimento creato manualmente. I domini di riferimento sono:

- `en.wikipedia.org`;
- `viaggi-usa.it`;
- `cbsnews.com`;
- `cnbc.com`.

Per gestire in modo efficiente l'estrazione da queste sorgenti, il progetto è stato strutturato seguendo un'architettura a microservizi (come si può osservare nella Figura 3.1), separando la logica di estrazione, valutazione e salvataggio dei dati (*Backend*) dall'interfaccia utente (*Frontend*). L'intero sistema è containerizzato mediante Docker, tramite le specifiche definite nel file `docker-compose.yaml` per garantire la portabilità e riproducibilità dell'ambiente di esecuzione. Docker è infatti una piattaforma *open source* per lo sviluppo, la distribuzione e l'esecuzione delle applicazioni. Consente di separare queste ultime dall'infrastruttura grazie all'utilizzo dei *container*, ambienti leggeri e isolati che racchiudono tutto il necessario per eseguire un'applicazione in modo indipendente dal sistema host [3]. Nello specifico del framework, l'orchestrazione tramite Compose ha permesso di avviare simultaneamente e far comunicare i moduli di estrazione, il database relazionale, il motore di inferenza Ollama per l'esecuzione del modello linguistico locale e l'interfaccia utente. L'intero codice sorgente del progetto è disponibile pubblicamente come anticipato in precedenza su GitHub.

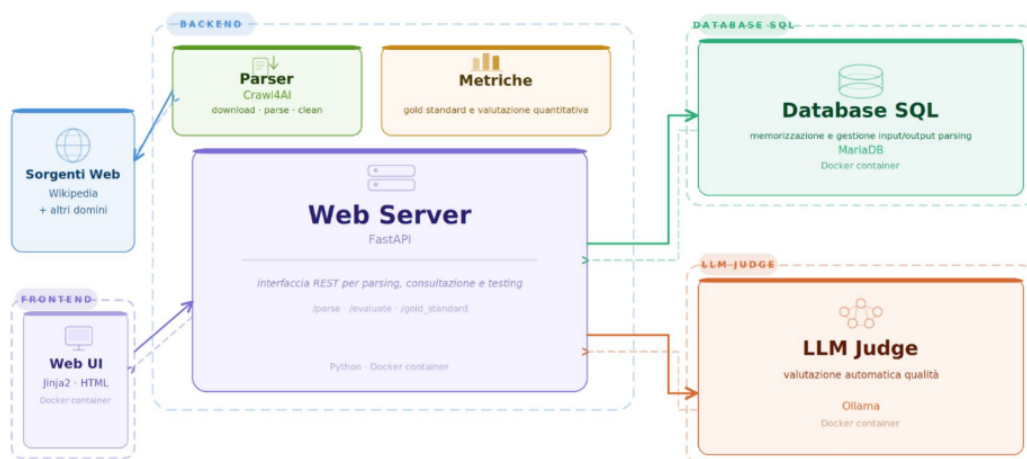


Figura 3.1. Architettura a microservizi del progetto. Lo schema illustra la comunicazione tra le varie componenti del sistema.

3.2 Progettazione del backend

Il *backend*, esposto sulla porta 8003, è stato sviluppato seguendo un'architettura modulare in Python e si articola nei seguenti macro-componenti fondamentali:

- **Validazione dati e server (`server.py`):** questo modulo espone i servizi verso il *frontend* sfruttando il framework FastAPI. Implementa l'interfaccia REST API e utilizza modelli Pydantic per una validazione rigorosa dei dati in ingresso e in uscita, assicurando che ogni endpoint (`/parse`, `/domains`, `/gold_standard`, ecc.) rispetti le specifiche richieste. La logica di *routing* smista dinamicamente le richieste ai diversi *parser* in base al dominio dell'URL fornito.
- **Motore di estrazione:** ognuno dei quattro domini è gestito da un modulo dedicato che implementa una *pipeline* di *parsing* a tre stadi basata sulla libreria Crawl4AI:
 1. *Configurazione e acquisizione:* viene utilizzato `BrowserConfig` in modalità *headless* e `CrawlerRunConfig` con `CacheMode.BYPASS` per garantire l'acquisizione in tempo reale del contenuto. In alternativa, fornendo in input una stringa HTML, permette l'estrazione statica diretta senza interrogazione web.
 2. *DOM cleaning tramite JavaScript:* prima della conversione in Markdown, viene iniettato uno script JS mirato per manipolare il DOM e rimuovere selettivamente elementi di disturbo come infobox, sidebar, footer e banner.
 3. *Post-processing regex:* il testo convertito in Markdown subisce un ulteriore filtraggio tramite espressioni regolari per eliminare residui di tag, metadati, riferimenti bibliografici e spazi bianchi ridondanti, restituendo un testo pulito pronto per l'analisi.

- *Calcolo delle metriche (**accuracy_tester.py**)*: rappresenta il motore di valutazione dell'intero sistema. È stato progettato come un servizio indipendente che riceve stringhe di testo (estratto e di riferimento) e calcola il grado di sovrapposizione tra i set di token, operando una normalizzazione preventiva (*lowercase* e rimozione della punteggiatura) per garantire l'oggettività del confronto.

Completano l'architettura *backend* gli *script* ausiliari `init_db.py` per l'inizializzazione della base di dati, `remove_markdown.py` per la rimozione finale del Markdown e `clean_html.py` per la pulizia dell'html grezzo necessaria all'*LLM-as-a-Judge* in configurazione *Reference-Free*.

3.2.1 Gestione degli endpoint

Il cuore comunicativo del *backend* è gestito da FastAPI, un framework *web* moderno e ad alte prestazioni per la creazione di API in Python, basato sulla tipizzazione standard *type hints*. I principali punti di forza di questo strumento risiedono nella sua robustezza, nella facilità di sviluppo e nella capacità di generare automaticamente una documentazione interattiva [4]. Il framework espone i servizi attraverso un'architettura RESTful e garantisce la validazione di ogni richiesta e risposta tramite i modelli *Pydantic*. Questi ultimi prevengono la creazione di dati malformati imponendo formati specifici (come `ParseRequest` o `EvaluateRequest`) e assicurano la generazione di JSON per una facile integrazione con gli altri componenti del sistema [5].

Le rotte dell'API possono essere raggruppate in tre macro-categorie:

- **Servizi di estrazione (`/parse`)**: l'endpoint principale accetta richieste POST contenenti l'URL da pulire. Il dominio viene analizzato e instradato dinamicamente al *parser* di riferimento per l'elaborazione. Una variabile booleana consente di verificare preliminarmente la presenza dell'HTML nel database locale: se `local=True` il *parsing* avviene off-line, riducendo drasticamente i tempi di latenza e le richieste di rete, altrimenti viene scaricato l'HTML originale.
- **Servizi di valutazione (`/evaluate` e `/evaluate_judge`)**: questi endpoint costituiscono il motore di analisi quantitativa. `/evaluate` esegue la pulizia del Markdown (`remove_markdown`) e restituisce le metriche deterministiche e semantiche calcolate dal modulo `accuracy_tester.py`. Le rotte `/evaluate_judge` e `/evaluate_judge_no_gs` instaurano invece una comunicazione asincrona, tramite la libreria `httpx` [6], con il container di Ollama, interrogando l'LLM con i *prompt* opportunamente ingegnerizzati e imponendo al modello di restituire un output rigorosamente in formato JSON, contenente il voto numerico e il *feedback* qualitativo.
- **Gestione dello stato e statistiche (`/db_stats`, `/full_gs_eval`)**: un set di endpoint dedicati al monitoraggio e all'aggregazione dei dati. Permettono al *frontend* di calcolare le medie delle metriche per ogni dominio, interrogare e salvare i dati nel database e gestire le operazioni CRUD (Create, Read,

Update, Delete) sulle risorse web memorizzate e sui *gold standard*. Completa il modulo la rotta `/status` per il controllo dell'attività del *backend*, dell'integrità del *database* e della raggiungibilità del modello linguistico.

Nel Listato 3.1 è riportato un frammento di codice che mostra i modelli di validazione, inclusa la struttura annidata della risposta e l'endpoint dedicato al calcolo delle metriche di valutazione:

Listing 3.1. Definizione dei modelli Pydantic e dell'endpoint per la valutazione quantitativa.

```

1  # Modello per le metriche standard
2  class TokenLevelEval(BaseModel):
3      precision: float
4      recall: float
5      f1: float
6  # Modello per le altre metriche
7  class XEval(BaseModel):
8      jaccard_similarity: float
9      bigram_overlap: float
10     cosine_similarity: float
11     meteor: float
12     bert_score: float
13
14 # Modelli di Input/Output per l'endpoint /evaluate
15 class EvaluateRequest(BaseModel):
16     parsed_text: str
17     gold_text: str
18
19 class EvaluateResponse(BaseModel):
20     token_level_eval: TokenLevelEval
21     x_eval: XEval
22
23 # Endpoint per il calcolo delle metriche
24 @app.post("/evaluate", response_model=EvaluateResponse)
25 async def evaluate_text(data: EvaluateRequest):
26
27     # Rimozione del Markdown prima della valutazione
28     normalized_parsed = remove_markdown(data.parsed_text)
29     normalized_gold = remove_markdown(data.gold_text)
30
31     # Chiamata al motore di calcolo indipendente
32     ris = calculate_metrics(normalized_parsed, normalized_gold)
33
34     return {
35         "token_level_eval": TokenLevelEval(
36             precision=ris["precision"],
37             recall=ris["recall"],
38             f1=ris["f1"]
39         ),
40         "x_eval": XEval(
41             jaccard_similarity=ris["jaccard_similarity"],
42             bigram_overlap=ris["bigram_overlap"],
43             cosine_similarity=ris["cosine_similarity"],
44             meteor=ris["meteor"],
45             bert_score=ris["bert_score"]
46     )

```

3.2.2 Logica dei parser

La fase di estrazione e pulizia dei dati è affidata a script specializzati per singolo dominio, implementati sfruttando la libreria asincrona *Crawl4AI*. Questo strumento *open-source* è stato ideato per facilitare l'integrazione con i modelli linguistici e le *pipeline* RAG, generando nativamente testo in formato Markdown [7]. Nonostante i selettori CSS e le espressioni regolari varino a seconda del dominio, la logica generale dei *parser* rimane invariata.

L'estrazione si articola in tre fasi sequenziali:

1. **Configurazione e acquisizione:** impostazione del *browser headless* e del generatore Markdown, con regole restrittive per ignorare link e immagini e isolare esclusivamente la classe con i contenuti rilevanti della pagina, al fine di ridurre il rumore.
2. **Manipolazione del DOM:** iniezione di codice JavaScript eseguito direttamente nel *browser* prima dell'estrazione. Questo passaggio è essenziale per rimuovere alla radice elementi strutturali non informativi specifici per ogni dominio (come *sidebar*, *footer* e banner pubblicitari) prima che vengano convertiti in testo.
3. **Filtraggio tramite regex:** raffinamento finale sul testo Markdown generato, per eliminare elementi di disturbo residui come tag di formattazione o metadati non catturati dai selettori CSS.

Il flusso logico è descritto in Figura 3.2.

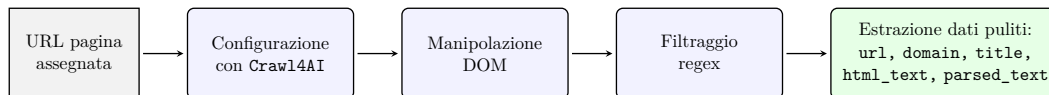


Figura 3.2. Pipeline sequenziale di estrazione dati.

A titolo esemplificativo, nel Listato 3.2 è riportata l'implementazione del *parser* dedicato al dominio di Wikipedia. Il codice evidenzia la logica di rimozione delle *infobox* tramite JavaScript e la successiva pulizia dei tipici tag redazionali (es. [edit], [citation needed]) tramite espressioni regolari.

Listing 3.2. Implementazione del *parser* asincrono per il dominio Wikipedia.

```

1 from crawl4ai import AsyncWebCrawler, BrowserConfig, CrawlerRunConfig
  , CacheMode, DefaultMarkdownGenerator
2 import re
3
4 async def parser_wiki(url: str, html_text: str) -> dict:
5
6     # Configurazione del browser
7     browser_config = BrowserConfig(headless=True)
  
```

```

8
9  # Configurazione del Markdown Generator
10 md_generator = DefaultMarkdownGenerator(
11     options={
12         "ignore_links": True,
13         "escape_html": False,
14         "ignore_images": True
15     }
16 )
17
18 # Iniezione JavaScript per la pulizia del DOM
19 remove_infobox_js = """
20     const infoboxes = document.querySelectorAll('.infobox, .
21     sidebar, .vertical-navbox, .navbox, .portal, .toc, .metadata, .
22     ambox, .hatnote, .shortdescription, .figcaption, .thumbcaption');
23     infoboxes.forEach(box => box.remove());
24     const figures = document.querySelectorAll('figure');
25     figures.forEach(f => f.remove());
26     """
27
28 css_list = ["h1#firstHeading", "#mw-content-text"]
29
30 # Configurazione del Crawler
31 crawler_config = CrawlerRunConfig(
32     cache_mode=CacheMode.BYPASS,
33     target_elements=css_list,
34     markdown_generator=md_generator,
35     js_code=remove_infobox_js
36 )
37
38 # Esecuzione del crawler da web o da HTML locale
39 async with AsyncWebCrawler(config=browser_config) as crawler:
40     if html_text == '':
41         result = await crawler.arun(url=url, config=
42         crawler_config)
43     else:
44         result = await crawler.arun(url=f"raw:{html_text}",
45         config=crawler_config)
46
47 # Pulizia del testo
48 clean_text = re.sub(r"[[a-z]]/[d+]/[edit]/[show]/[
49 update]/[s*]", "", result.markdown)
50 clean_text = re.sub(r'(\*|_|)(.*?)1', r'\2', clean_text)
51 clean_text = re.sub(r"[[citation needed]/[clarification needed
52 /][supporting]/[A-Z]+]", "", clean_text)
53 clean_text = re.sub(r"^\[-:s/]+\n?", "", clean_text, flags=re
54 .MULTILINE)
55 clean_text = re.sub(r'^.*Main article[s]?s*.*$', '',
56 clean_text, flags=re.MULTILINE | re.IGNORECASE)
57 clean_text = re.sub(r"s*/s*", " ", clean_text)
58 clean_text = re.sub(r"\*/\*", "", clean_text)
59 text = re.split(r"##s*(?:See also/References/Notes/Further
60 reading/External links)", clean_text, flags=re.IGNORECASE)
61 clean_text = text[0].strip()
62
63 # Creazione del JSON
64 title_match = re.search(r'<title>(.*?)</title>', result.html, re.

```

```

56     IGNORECASE)
57     page_title = title_match.group(1).replace(" - Wikipedia", "") if
58     title_match else "title_not_found"
59
60     return {
61         "url": url,
62         "domain": "en.wikipedia.org",
63         "title": page_title,
64         "html_text": result.html,
65         "parsed_text": clean_text
66     }

```

3.2.3 Motore di calcolo delle metriche

La valutazione quantitativa dell'accuratezza nell'estrazione delle informazioni operata dai *parser* è realizzata da un modulo indipendente. Questo componente, denominato `accuracy_tester.py`, funge da vero e proprio motore di calcolo: riceve in ingresso sia il testo pulito che il rispettivo testo di riferimento (*gold standard*), compilato accuratamente a mano. L'algoritmo converte i testi in due insiemi distinti di parole per il calcolo delle metriche tradizionali, elaborando contestualmente i testi interi per l'estrazione delle metriche semantiche vettoriali. Successivamente li confronta e restituisce un dizionario strutturato con le valutazioni complete, gestendo opportunamente tutti i casi limite e le eccezioni, come la ricezione di stringhe vuote o il verificarsi di divisioni per zero.

L'implementazione utilizza alcune librerie standard per l'elaborazione del linguaggio naturale e l'apprendimento automatico, tra cui *NLTK* per la metrica METEOR, *Scikit-learn* per la vettorializzazione TF-IDF e il calcolo della *Cosine similarity*, e *BERTScore* per la valutazione semantica. Un dettaglio rilevante riguarda proprio quest'ultima metrica: per rispettare i vincoli di memoria RAM dell'infrastruttura, il calcolo di BERTScore è stato realizzato con l'uso del modello alleggerito `distilbert-base-uncased`, così da garantire un ottimo compromesso tra accuratezza semantica e prestazioni computazionali.

Nel Listato 3.3 è riportato il cuore della logica algoritmica del modulo, omettendo per brevità i blocchi di gestione delle eccezioni e dei casi limite (come la ricezione di stringhe vuote).

Listing 3.3. Logica di estrazione dei token e calcolo delle metriche di valutazione.

```

1  import re
2  from nltk.translate.meteor_score import meteor_score
3  from sklearn.feature_extraction.text import TfidfVectorizer
4  from sklearn.metrics.pairwise import cosine_similarity
5  from bert_score import score
6
7  def calculate_metrics(parsed_text: str, gold_text: str) -> dict:
8
9      # ... [Omissioni: Gestione stringhe vuote e sanitizzazione] ...
10
11     # Estrazione dei token tramite regex
12     extracted_words = re.findall(r'\b\w+\b', parsed_text.lower())
13     gs_words = re.findall(r'\b\w+\b', gold_text.lower())
14     extracted_tokens = set(extracted_words)

```

```

15     gs_tokens = set(gs_words)
16
17     if not extracted_tokens or not gs_tokens:
18         return {
19             "precision": 0.0,
20             "recall": 0.0,
21             "f1": 0.0,
22             "jaccard_similarity": 0.0,
23             "bigram_overlap": 0.0,
24             "cosine_similarity": 0.0,
25             "meteor": 0.0,
26             "bert_score": 0.0
27         }
28
29     # Metriche tradizionali basate sugli insiemi
30     intersection = extracted_tokens & gs_tokens
31     union = extracted_tokens | gs_tokens
32
33     precision = len(intersection) / len(extracted_tokens)
34     recall = len(intersection) / len(gs_tokens)
35     f1 = (2 * precision * recall) / (precision + recall) if (
36         precision + recall) > 0 else 0.0
37
38     # Metriche avanzate
39     jaccard_similarity = len(intersection) / len(union) if len(union)
40         > 0 else 0.0
41
42     # Creazione di insiemi di coppie di parole consecutive per Bigram
43     overlap
44     ex_bigrams = set(zip(extracted_words[:-1], extracted_words[1:]))
45     gs_bigrams = set(zip(gs_words[:-1], gs_words[1:]))
46     bigram_overlap = len(ex_bigrams & gs_bigrams) / min(len(
47         ex_bigrams), len(gs_bigrams)) if ex_bigrams and gs_bigrams else
48         0.0
49
50     # Cosine similarity con vettorializzazione TF-IDF
51     vectorizer = TfidfVectorizer(token_pattern=r'(?u)\b\w+\b')
52     tfidf_matrix = vectorizer.fit_transform([gold_text, parsed_text])
53     cos_similarity = cosine_similarity(tfidf_matrix[0:1],
54         tfidf_matrix[1:2])[0][0]
55
56     # METEOR e BERTScore (modello distilbert per ottimizzazione RAM)
57     try:
58         meteor = meteor_score([gs_words], extracted_words)
59     except Exception:
60         meteor = 0.0
61
62     try:
63         P, R, F1_bert = score([parsed_text], [gold_text], lang="en",
64             model_type="distilbert-base-uncased", verbose=False)
65         bert_score = F1_bert.item()
66     except Exception as e:
67         print(f"Error BERTScore: {e}")
68         bert_score = 0.0
69
70     return {
71         "precision": float(precision),

```

```
65     "recall": float(recall),
66     "f1": float(f1),
67     "jaccard_similarity": float(jaccard_similarity),
68     "bigram_overlap": float(bigram_overlap),
69     "cosine_similarity": float(cos_similarity),
70     "meteor": float(meteor),
71     "bert_score": float(bert_score)
72 }
```

3.3 Progettazione del frontend

L'interfaccia utente, esposta sulla porta 8004, è stata progettata per fornire un punto di accesso intuitivo alle funzionalità del sistema. Si compone dei seguenti tre moduli principali:

- **Gestione delle richieste (frontend.py):** sviluppato anch'esso in FastAPI, questo modulo agisce come un'interfaccia tra l'utente e il backend. Utilizza il client asincrono `httpx` per comunicare internamente con i servizi del *backend*. Recupera dinamicamente la lista dei domini supportati e degli URL presenti nel database per popolare l'interfaccia all'avvio. Interroga i *gold standard* e monitora in tempo reale lo stato dei componenti (Backend, MariaDB e Ollama). Gestisce l'intero ciclo di richiesta-risposta che comprende tre flussi principali: l'analisi di URL singoli coordinando l'estrazione del testo per qualsiasi URL compatibile e, qualora la risorsa sia presente nel dataset di riferimento, innescando automaticamente il recupero e la visualizzazione delle metriche di valutazione; un pannello di amministrazione CRUD per la gestione interattiva dei dati di riferimento ed infine una *dashboard* statistica per monitorare facilmente i risultati.
- **Motore di rendering (templates/):** la presentazione dei dati è affidata a Jinja2 per la generazione dinamica di pagine HTML, stilizzate tramite CSS dedicato. Questo motore è fondamentale poiché consente di scrivere codice simile alla sintassi Python grazie a dei *placeholders* direttamente all'interno del documento, garantendo velocità ed espressività [8]. L'interfaccia è in grado di adattarsi ai dati ricevuti, mostra messaggi di errore contestuali e si articola in diverse aree specializzate: una *home page* di riepilogo dello stato, un'interfaccia di parser con tre colonne per facilitare il confronto testuale tra HTML grezzo, *gold standard* e testo estratto, esponendo contestualmente sotto le relative metriche dell'URL preso in input. Sono inoltre presenti un pannello per l'inserimento o rimozione di risorse web e una pagina analitica per vedere i risultati complessivi di ciascun dominio, sia metriche base e avanzate che giudizio dell'LLM.
- **Stile (static/):** questa directory contiene il foglio di stile CSS che definisce l'estetica dell'applicazione.

3.3.1 Architettura API e gestione delle latenze

Il cuore logico dell'interfaccia utente risiede nello script `frontend.py` che funge da *API gateway* tra le interazioni dell'utente e i servizi del *backend*, oltre a coordinare il caricamento dei file statici e dei *template* HTML tramite Jinja2. Dal punto di vista ingegneristico, una delle sfide principali di questo modulo riguarda la gestione delle latenze. Infatti, operazioni come l'estrazione del testo, il calcolo delle metriche e, soprattutto, la chiamata del modello linguistico (*LLM-as-a-Judge*), possono richiedere una grande quantità di tempo, persino minuti. Per evitare di bloccare il *thread* principale del server durante l'attesa delle risposte e continuare a garantire il normale funzionamento del sistema, le comunicazioni HTTP interne sono state implementate in modo interamente asincrono sfruttando la libreria `httpx` e il paradigma `async/await`.

Nel Listato 3.4 è riportata una sintesi dell'architettura del modulo, focalizzata sulla rotta di analisi principale. Si può notare l'impostazione di un *timeout* esteso e l'uso di *context manager* asincroni per garantire una corretta chiusura delle connessioni di rete anche in caso di eccezioni.

Listing 3.4. Inizializzazione dell'applicazione ed esempio di gestione asincrona delle richieste nel frontend.

```

1 from fastapi import FastAPI, Request, Form
2 from fastapi.templating import Jinja2Templates
3 from fastapi.staticfiles import StaticFiles
4 import httpx
5
6 app = FastAPI(title="Frontend progetto")
7
8 # Montaggio risorse statiche e template
9 app.mount("/static", StaticFiles(directory="static"), name="static")
10 templates = Jinja2Templates(directory="templates")
11 BACKEND_URL = "http://backend:8003"
12
13 @app.post("/analyze")
14 async def analyze_url(request: Request, url: str = Form(...), mode:
15     str = Form("live")):
16     domains, gs_urls = await get_home_data()
17
18     parsed_data = None
19     eval_data = None
20     gold_text = None
21     judge_data = None
22     error = None
23     is_local = (mode == "local")
24
25     # Client HTTP asincrono con timeout esteso
26     async with httpx.AsyncClient(timeout=1800.0) as client:
27         try:
28             # Richiesta al backend per il parsing
29             parse_resp = await client.post(f"{BACKEND_URL}/parse",
30                 json={"url": url, "local": is_local})
31
32             if parse_resp.status_code == 200:
33                 parsed_data = parse_resp.json()

```

```

33         # Recupero del gold standard
34         gs_resp = await client.get(f"{BACKEND_URL}/
gold_standard", params={"url": url})
35
36         if gs_resp.status_code == 200:
37             gold_text = gs_resp.json().get("gold_text")
38             # Calcolo delle metriche
39             eval_resp = await client.post(f"{BACKEND_URL}/
evaluate", json={
40                 "parsed_text": parsed_data["parsed_text"],
41                 "gold_text": gold_text
42             })
43             if eval_resp.status_code == 200:
44                 eval_data = eval_resp.json()
45
46             # Chiamata all'LLM
47             judge_resp = await client.post(f"{BACKEND_URL}/
evaluate_judge", json={
48                 "parsed_text": parsed_data["parsed_text"],
49                 "gold_text": gold_text
50             })
51             if judge_resp.status_code == 200:
52                 judge_data = judge_resp.json()
53
54             # Chiamata all'LLM reference-free
55             judge_no_gs_resp = await client.post(f"{
BACKEND_URL}/evaluate_judge_no_gs", json={
56                 "parsed_text": parsed_data["parsed_text"],
57                 "html_text": parsed_data["html_text"]
58             })
59             if judge_no_gs_resp.status_code == 200:
60                 judge_no_gs_data = judge_no_gs_resp.json()
61                 if judge_data is None:
62                     judge_data = {}
63
64                 judge_data["judge_score_no_gs"] =
judge_no_gs_data.get("judge_score")
65                 judge_data["judge_feedback_no_gs"] =
judge_no_gs_data.get("judge_feedback")
66             else:
67                 if judge_data is None:
68                     judge_data = {}
69                 judge_data["judge_score_no_gs"] = 0
70                 judge_data["judge_feedback_no_gs"] = f"ERRORE
DI SISTEMA: {judge_no_gs_resp.status_code} - {judge_no_gs_resp.
text}"
71             else:
72                 error = parse_resp.json().get("detail", f"Error in
backend: {parse_resp.status_code}")
73
74             except Exception as e:
75                 error = f"Error in communication with the server: {str(e)}
}"
76
77         # Immissione dei risultati nel motore Jinja2
78         return templates.TemplateResponse(
79             request=request,

```

```

80     name="analyze.html",
81     context={
82         "domains": domains,
83         "gs_urls": gs_urls,
84         "parsed_data": parsed_data,
85         "gold_text": gold_text,
86         "eval_data": eval_data,
87         "judge_data": judge_data,
88         "error": error,
89         "selected_mode": mode
90     }
91 )

```

3.3.2 Accesso al sistema e interfaccia di analisi

Il punto di accesso dell'interfaccia utente è rappresentato dalla *home page* (`home.html`), progettata per offrire una panoramica immediata sulle funzionalità del sistema e sullo stato di salute dei servizi sottostanti. Questa prima schermata (Figura 3.3) permette di scegliere tra tre moduli principali di navigazione (*Parser & Evaluation*, *Gold Standard* e *Stats*) e presenta indicatori visivi per il monitoraggio in tempo reale dell'attività dei vari componenti architetturali (*Backend*, *MariaDB* e *Ollama*). Inoltre la pagina elenca i quattro domini attualmente supportati.

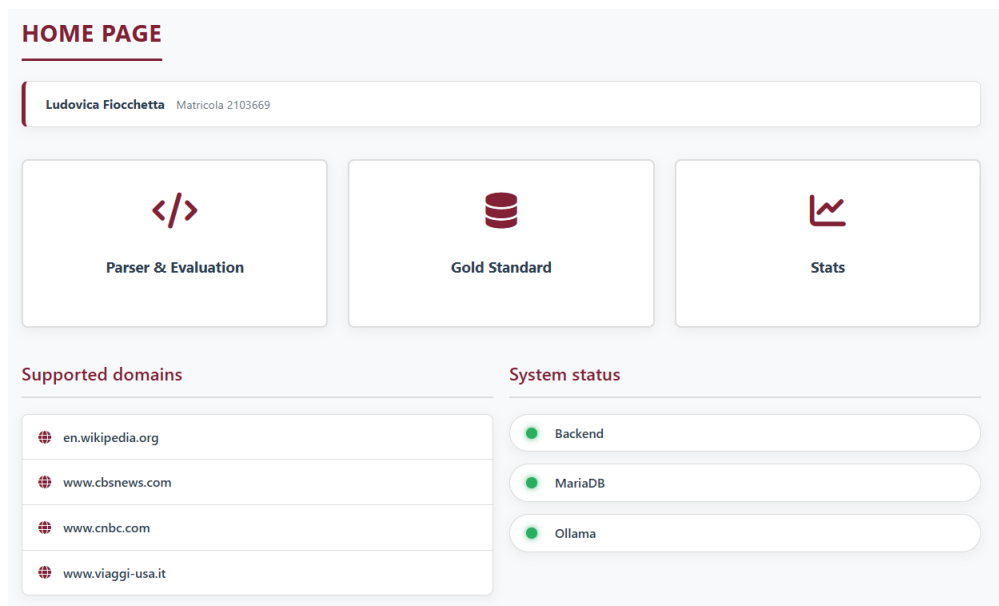


Figura 3.3. Home page: monitoraggio dei servizi.

Accedendo alla sezione di *Parser & Evaluation*, il cuore operativo dell'applicativo web è rappresentato dal *template analyze.html*. L'obiettivo di quest'interfaccia, esposta nella Figura 3.4 è fornire all'utente uno strumento visivo facile e immediato per la valutazione della qualità dell'estrazione. L'utente ha a disposizione due flussi di lavoro principali: può inserire manualmente un URL arbitrario, di uno dei quattro domini supportati, da parsare in tempo reale (*Live mode*), eliminando però

la possibilità di confronto con il *gold standard*. In alternativa, tramite un menù a tendina popolato dinamicamente con tutti gli indirizzi presenti nel database, può selezionarne uno e decidere in quale modalità utilizzare l'HTML (*Local mode* o *Live mode*). Inoltre, l'interfaccia garantisce un'opportuna gestione di tutti i casi limite.

PARSER & EVALUATION ← Back to Home

Test URL

https://it.wikipedia.org/wiki/Intelligenza_artificiale

-- Or select from Gold Standard --

Mode

Live Local

Download the page in real-time from the web Use already saved HTML in the database

EXECUTE PARSING AND EVALUATION

Figura 3.4. Interfaccia di analisi.

A seguito dell'elaborazione, il *layout* reattivo della pagina si espande per ospitare i testi analizzati. Come illustrato in Figura 3.5, questa scelta permette un confronto visivo diretto tra il codice HTML sorgente scaricato, il testo pulito estratto dal *parser* e, in un riquadro dedicato, il *gold standard* di riferimento recuperato dal database, se disponibile. Un apposito messaggio di notifica conferma all'utente il corretto completamento della valutazione.

Input HTML

```
<!DOCTYPE html><html lang="en-US" class="
  theme--responsive
  device--type-desktop
  device--platform-web
  device--size-
  content--type-article
  content--slug-coffee-tea-caffeine-dementia-risk-study
  page--type-news-item
  is--show
  show--name=healthwatch

  edition--us

  has-story-redesign
```

Output parsed

What to know about a new study on coffee, tea, caffeine and dementia risk

A long-term analysis shows that people who drink caffeinated coffee and tea have a lower risk of developing dementia, although the study does not prove cause and effect.

Dr. Céline Gounder, CBS News medical contributor and editor-at-large for public health at KFF Health News, warned that the results should be taken "with a massive grain of salt."

The research, published in the medical journal JAMA on Monday, analyzed data on 131,821 people who had participated in two studies that ran from 1986 to 2023. Of that group, 11,033 developed dementia during the period studied.

The researchers compared each participant's self-reported intake of coffee, caffeinated tea and decaffeinated coffee, and tracked records of their cognitive health over time. They found that men and women who drank the most caffeinated coffee had an 18% lower risk of dementia when compared to those who reported little or no

Gold text

What to know about a new study on coffee, tea, caffeine and dementia risk

A long-term analysis shows that people who drink caffeinated coffee and tea have a lower risk of developing dementia, although the study does not prove cause and effect.

Dr. Céline Gounder, CBS News medical contributor and editor-at-large for public health at KFF Health News, warned that the results should be taken "with a massive grain of salt."

The research, published in the medical journal JAMA on Monday, analyzed data on 131,821 people who had participated in two studies that ran from 1986 to 2023. Of that group, 11,033 developed dementia during the period studied.

The researchers compared each participant's self-reported intake of coffee, caffeinated tea and decaffeinated coffee, and tracked records of their cognitive health over time. They found that men and women who drank the most caffeinated coffee had an 18% lower risk of dementia when compared to those who reported little or no caffeinated coffee consumption. Caffeinated coffee drinkers also showed a lower prevalence of cognitive decline, the researchers said.

The benefits appeared to be most pronounced in people who drank two to three cups of caffeinated coffee a day, the researchers said, or one to two cups of caffeinated tea.

"Higher caffeinated coffee intake was significantly associated with lower risk of dementia," they wrote. "Decaffeinated coffee intake was n

✓ URL is in the Gold Standard, evaluation completed.

Figura 3.5. Interfaccia di confronto: visualizzazione affiancata dell'HTML sorgente, del testo estratto e del *gold standard* per un articolo medico di *cbsnews*.

Nella porzione inferiore della stessa schermata, mostrata in Figura 3.6, il motore di *templating* Jinja2 inserisce dinamicamente i risultati numerici restituiti dal *backend*. L'utente ha così accesso a uno schema riassuntivo diviso in sezioni: metriche tradizionali (Precision, Recall, F1-Score), valutazioni avanzate (Cosine similarity, BERTScore e METEOR) e i due giudizi formulati dall'LLM (con e senza referenza), entrambi quantitativi e qualitativi.

STANDARD METRICS

Precision	Recall	F1 score
1.00	1.00	1.00

ADVANCED METRICS

Jaccard similarity	Bigram overlap	Cosine similarity	Meteor	Bertscore
1.00	1.00	1.00	1.00	1.00

LLM JUDGE ASSESSMENT WITH GOLD STANDARD

Score	Feedback
5/5	Il testo estratto è perfettamente coerente e accurato, con la stessa struttura e contenuto del testo di riferimento.

LLM JUDGE ASSESSMENT REFERENCE-FREE

Score	Feedback
4/5	Il testo estratto cattura bene il contenuto informativo principale della pagina web, ma manca alcuni dettagli specifici come la citazione di Dr. Céline Gounder e l'elenco delle conclusioni del studio. L'estrazione è coerente con l'HTML e rappresenta un buon lavoro.

Figura 3.6. Metriche di valutazione del precedente articolo di *cbsnews*.

3.3.3 Pannello CRUD e dashboard statistica

A supporto del motore di calcolo, l'interfaccia fornisce due ulteriori moduli per l'amministrazione del sistema e la valutazione aggregata, rispettivamente `gs_manage.html` e `stats.html`.

Il pannello di gestione dei (*Gold Standard*) espone un'interfaccia CRUD completa, illustrata in Figura 3.7, che consente all'utente di popolare, aggiungere ed eliminare testi di riferimento nel database senza dover interagire direttamente con quest'ultimo. Attraverso form dedicati, è possibile scaricare l'HTML grezzo di una nuova pagina web, inserire manualmente il contenuto e salvarlo stabilmente nel sistema, oppure rimuovere risorse obsolete.

La *dashboard* statistica, infine, aggrega i risultati delle singole valutazioni chiamando la rotta `/full_gs_eval` per restituire una panoramica prestazionale divisa per dominio di appartenenza. Questa è fondamentale per valutare la flessibilità dei *parser* su layout diversi: calcolando le medie dei punteggi per ogni specifico URL analizzato, l'interfaccia permette di individuare facilmente dove l'algoritmo di estrazione necessita di ulteriore raffinamento. La schermata permette, inoltre, un resoconto quantitativo immediato indicando il numero totale di risorse presenti nella base di dati.

GOLD STANDARD BUILDER ← Back to Home

✓ HTML downloaded.

Supported domains: en.wikipedia.org | URL: https://en.wikipedia.org/wiki/Chocolate | **FETCH HTML**

HTML downloaded | Copy HTML

```
<!DOCTYPE html><html class="client-js vector-feature-language-in-header-enabled vector-
feature-language-in-main-menu-disabled vector-feature-language-in-main-page-header-disabled
vector-feature-page-tools-pinned-disabled vector-feature-main-menu-pinned-disabled vector-
feature-limited-width-clientpref-1 vector-feature-limited-width-content-enabled vector-
feature-custom-font-size-clientpref-1 skin-theme-clientpref-day vector-too-available skin-
thumbsie-clientpref-standard vector-animations-ready vector-feature-too-pinned-clientpref-0
vector-feature-appearance-pinned-clientpref-0 ve-available" lang="en" dir="ltr"><head>
<meta charset="UTF-8">
<title>Chocolate - Wikipedia</title>
<script>(function() {var className="client-js vector-feature-language-in-header-enabled vector-
feature-language-in-main-menu-disabled vector-feature-language-in-main-page-header-disabled
vector-feature-page-tools-pinned-disabled vector-feature-main-menu-pinned-disabled vector-
feature-limited-width-clientpref-1 skin-theme-clientpref-day vector-sticky-header-enabled vector-
feature-main-menu-pinned-disabled vector-feature-limited-width-clientpref-1 vector-feature-
limited-width-content-enabled vector-feature-custom-font-size-clientpref-1 vector-feature-
appearance-pinned-clientpref-1 skin-theme-clientpref-day vector-sticky-header-enabled vector-
too-available skin-thumbsie-clientpref-standard";var cookie=document.cookie.match(/(?:^|;
)enwikimobileclientpreferences=([^\s;]+)/);if(cookie){cookie[1].split("%20").forEach(function(pref)
{className=className.replace(new RegExp('('+pref.replace(/-/>

```

Gold text

Paste here the text from the page...

SAVE TO DATABASE **CANCEL**

URL present in the GS

URL	Actions
https://en.wikipedia.org/wiki/Donald_Trump	Delete
https://en.wikipedia.org/wiki/Artificial_intelligence	Delete
https://en.wikipedia.org/wiki/Alfred_Romeo_159	Delete
https://en.wikipedia.org/wiki/Scoby-Doo	Delete
https://en.wikipedia.org/wiki/Lazarote	Delete
https://en.wikipedia.org/wiki/Charles_Darwin	Delete
https://en.wikipedia.org/wiki/D-1_(Sony)	Delete
https://en.wikipedia.org/wiki/Art_museum	Delete

10 results

Figura 3.7. Pannello CRUD per la gestione interattiva dei *gold standard*.

3.4 Gestione dei dati

Per garantire la persistenza dei risultati ottenuti e ottimizzare le risorse l'architettura fa uso di un database relazionale, che organizza i dati in tabelle (entità) composte da righe e colonne, integrato alla *pipeline* mediante MariaDB. MariaDB è uno dei più importanti sistemi di gestione di database relazionali *open-source* ed è un *fork* di MySQL, garantendone così altissima compatibilità e prestazioni elevate [9]. Questa scelta strutturale consente una maggiore velocità nello svolgimento delle valutazioni successive di una determinata pagina web poiché, grazie ad una strategia di *caching* locale, il sistema è in grado di riutilizzare l'HTML precedentemente salvato, evitando di doverlo scaricare nuovamente e ripetutamente dalla rete. Inoltre, per assicurare che il database non venga azzerato alla chiusura dei *container* Docker, i dati vengono mappati su una cartella persistente dedicata (`mariadb_data`).

3.4.1 Modello relazionale

La base di dati è stata progettata seguendo i principi di normalizzazione in modo da garantire la coerenza delle informazioni. Il centro dello schema è la risorsa web originaria, identificata univocamente dal proprio indirizzo URL, che svolge il ruolo

di chiave primaria e di chiave esterna per le entità collegate, come illustrato nel diagramma entità-relazione in Figura 3.8.

Lo schema relazionale è composto dalle seguenti quattro tabelle:

- **web_resources** (url, domain, title, html_text, created_at): è l'entità principale, salva l'HTML grezzo della pagina web, il titolo e il dominio di appartenenza. L'attributo `url` agisce da PRIMARY KEY.
- **gold_standard** (url, gold_text, created_at): memorizza il testo di riferimento associato ad un determinato indirizzo. È legata a **web_resources** tramite l'URL, impostato come chiave esterna (FOREIGN KEY) con logica di rimozione ON DELETE CASCADE.
- **parsed_pages** (url, parsed_text, created_at): questa tabella raccoglie il testo estratto dai *parser* in fase di esecuzione, ed è collegata sempre all'entità principale.
- **evaluation_results** (url, metrics_token_level, metrics_x_eval, judge_score, judge_feedback, judge_score_no_gs, judge_feedback_no_gs, evaluated_at): costituisce l'entità analitica del sistema. Memorizza tutte le valutazioni effettuate per un determinato URL: dalle metriche *token-level* (Precision, Recall, F1) e *x-eval* (Jaccard similarity, Bigram overlap, Cosine similarity, METEOR, BERTScore), fino al punteggio e al *feedback* qualitativo restituiti dall'LLM, sia con che senza *gold standard*. Anche questa è legata alla tabella centrale.

L'utilizzo del vincolo ON DELETE CASCADE assicura che l'eliminazione di una risorsa web dal pannello di amministrazione comporti la pulizia di tutte le valutazioni e i testi associati in modo automatico, mantenendo il database in uno stato coerente senza il bisogno di interrogazioni SQL multiple da parte del *backend*.

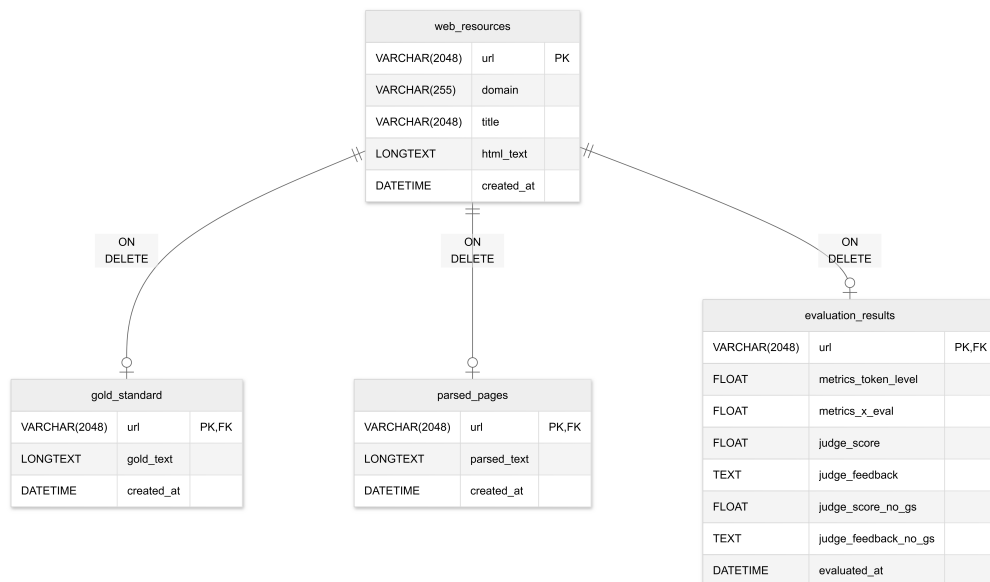


Figura 3.8. Schema ER del database.

3.4.2 Inizializzazione e popolamento

La creazione e il popolamento iniziale del database sono affidati ad uno script Python dedicato (`init_db.py`), eseguito automaticamente all'avvio del *container* del *backend*. La base di dati viene configurata al primo avvio del progetto, popolando di default le tabelle *web_resources*, e *gold_standard* leggendo i file JSON contenuti nella directory *gs_data/*. La tabella *evaluation_results* viene inizializzata impostando le metriche a NULL, dati che verranno aggiornati a seguito delle richieste di valutazione. Vengono inoltre modificate in seguito a richieste di *parsing* sia la tabella *web_resources* che *parsed_pages*. È anche presente un file di configurazione che elenca i domini di riferimento *domains.json*. Dal punto di vista ingegneristico una criticità tipica delle architetture a microservizi risiede nella desincronizzazione degli avvii: il *backend* potrebbe provare ad accedere al database prima che MariaDB abbia terminato la propria inizializzazione. Per risolvere questo problema, lo script implementa tentativi di connessione distanziati nel tempo prima di generare un'eccezione fatale. Una volta instaurata la connessione, le tabelle vengono create sovrascrivendo eventuali strutture preesistenti e ponendo attenzione a tutti i vincoli relazionali e d'integrità, come `INSERT IGNORE` che previene da errori in caso di inserimenti duplicati.

Nel Listato 3.5 sono riportati l'algoritmo di sincronizzazione della connessione e un estratto della generazione delle tabelle.

Listing 3.5. Sincronizzazione e definizione di *web_resources* e *parsed_pages*

```

1 import mariadb
2 import time
3 import sys
4
5 def get_db_connection(max_retries=5, delay=3):
6     """Garantisce la sincronizzazione tra i container gestendo i
7     ritardi di avvio."""
8     retries = 0
9     while retries < max_retries:
10         try:
11             conn = mariadb.connect(
12                 user="user",
13                 password="password",
14                 host="mariadb_service",
15                 port=3306,
16                 database="parser_db"
17             )
18             return conn
19         except mariadb.Error as e:
20             print(f"Tentativo {retries + 1} fallito. Errore MariaDB:
21             {e}")
22             retries += 1
23             time.sleep(delay)
24
25     print("Impossibile connettersi al database. Chiusura del servizio
26     .")
27     sys.exit(1)
28
29 def create_tables(cursor):
30     # 1. Creazione dell'entità forte web_resources

```

```
28     cursor.execute("""
29         CREATE TABLE IF NOT EXISTS web_resources (
30             url VARCHAR(2048) CHARACTER SET ascii COLLATE
31             ascii_general_ci PRIMARY KEY,
32             domain VARCHAR(255),
33             title VARCHAR(2048),
34             html_text LONGTEXT,
35             created_at DATETIME DEFAULT CURRENT_TIMESTAMP
36         )
37     """)
38
39     # 2. Creazione di un'entita debole parsed_pages con Foreign Key e
40     #    rimozione a cascata
41     cursor.execute("""
42         CREATE TABLE IF NOT EXISTS parsed_pages (
43             url VARCHAR(2048) CHARACTER SET ascii COLLATE
44             ascii_general_ci PRIMARY KEY,
45             parsed_text LONGTEXT,
46             created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
47             FOREIGN KEY (url) REFERENCES web_resources(url) ON DELETE
48             CASCADE
49         )
50     """)
51
52     # ... [Omesse: Creazione tabelle gold_standard e
53     #    evaluation_results e funzioni di popolamento, distruzione e set up
54     #    ] ...
```

Capitolo 4

Paradigma LLM-as-a-Judge

4.1 Inquadramento metodologico

Dopo aver delineato nel capitolo precedente l'infrastruttura alla base del sistema, questo capitolo ne esplora il nucleo innovativo. Come visto in precedenza, le metriche di valutazione tradizionali e avanzate per l'estrazione di testo dal web presentano limiti intrinseci nella comprensione del contesto e della reale utilità del dato parsato. Per superare questa rigidità, il progetto implementa il modello noto come *LLM-as-a-Judge*: un *Large Language Model* istruito tramite specifiche di *prompt engineering* giudica il testo estratto, assegnando uno *score* quantitativo e fornendo un *feedback* qualitativo. Sono state realizzate due varianti: una classica, che confronta il testo con il *gold standard* di riferimento, e una *reference-free*, basata esclusivamente sul paragone con l'HTML grezzo della pagina e sulle capacità di comprensione del modello stesso. A seguito di una fase di sperimentazione preliminare per individuare il miglior compromesso tra prestazioni e requisiti computazionali, il modello scelto per questo compito è Llama 3.2 nella versione da 3 miliardi di parametri (3B), eseguito localmente tramite Ollama. Il flusso del paradigma è esposto nella Figura 4.1.

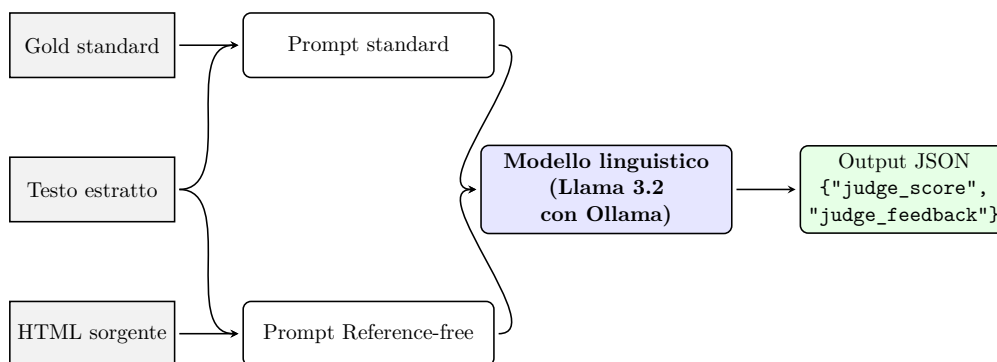


Figura 4.1. Diagramma che illustra il paradigma *LLM-as-a-Judge* sia nell'approccio classico che in quello *Reference-free*.

4.2 Integrazione del motore Ollama

L'esecuzione del modello Llama 3.2 avviene localmente per ragioni di *privacy*, controllo e riproducibilità. A livello architetturale, il modello è ospitato all'interno di un *container* Docker dedicato, gestito dal motore Ollama, che espone un'interfaccia API RESTful. Ollama è un *runner open-source* che permette di scaricare ed eseguire LLM, come Llama 3.2, sul proprio dispositivo tramite un server API locale [10]. Per ottimizzare il ciclo di vita dell'infrastruttura ed evitare il *download* dei pesi del modello linguistico di diversi gigabyte di dati a ogni riavvio del sistema, è stata configurata una directory apposita `ollama_data/`. Questa scelta garantisce che il modello rimanga memorizzato in modo permanente sul disco dell'*host*, in modo da ridurre notevolmente i tempi di configurazione del sistema. Il *backend* comunica con il servizio Ollama in modalità asincrona tramite la libreria `httpx`, inviando richieste POST all'*endpoint* di generazione. Per gestire efficacemente le tempistiche prolungate tipiche dell'inferenza basata su LLM su qualsiasi calcolatore, indipendentemente dalla quantità di RAM disponibile, sia il *client* che il *server* sono stati configurati con un *timeout* di 1800 secondi, così da prevenire l'interruzione prematura della connessione durante la generazione di giudizi complessi.

4.3 Ingegneria del prompt e gestione del contesto

L'efficacia del paradigma *LLM-as-a-Judge* ha una forte dipendenza con la modalità con cui il modello viene istruito a processare le informazioni. Per garantire stabilità ed efficienza durante l'inferenza, il sistema implementa una precisa gestione del contesto: l'input fornito al modello (sia il testo estratto che quello di riferimento) viene troncato ad un limite massimo di 1000 caratteri. Se il testo supera questa soglia, viene aggiunta l'annotazione `[...truncated text for time...]` per segnalare al modello la parzialità del dato, ottimizzando così i tempi di risposta e prevenendo errori di *timeout*. In questo modo il sistema riesce a funzionare perfettamente su calcolatori commerciali, prevenendo possibili problemi legati ai limiti della memoria RAM. Inoltre, tutta l'architettura è progettata per essere flessibile e scalabile: in presenza di macchine dotate di maggiore potenza computazionale, il troncamento può essere facilmente disabilitato, permettendo l'analisi dell'intera pagina. L'LLM è stato istruito per agire come un valutatore esperto di *web-parsing*, con il compito di assegnare un punteggio intero da 1 (estrazione completamente errata o mancante) a 5 (estrazione perfettamente coerente) e di generare un *feedback* qualitativo, rispondendo esclusivamente in formato JSON. Sono stati scritti due *prompt* distinti per gestire le due diverse modalità di valutazione. Nella modalità classica, il modello riceve in input sia il testo estratto dal *parser* che il *gold standard* corrispondente per confrontarli e giudicarli, come mostrato nel Listato 4.1.

Listing 4.1. Prompt per la modalità di valutazione tradizionale

```

1 @app.post("/evaluate_judge", response_model=EvaluateJudgeResponse)
2 async def evaluate_judge(data: EvaluateRequest):
3
4     char_limit = 1000
5
6     safe_parsed = data.parsed_text[:char_limit]
```

```

7     if len(data.parsed_text) > char_limit:
8         safe_parsed += "\n[...truncated text for time...]"
9
10    safe_gold = data.gold_text[:char_limit]
11    if len(data.gold_text) > char_limit:
12        safe_gold += "\n[...truncated text for time...]"
13
14    # prompt for the LLM-as-a-Judge
15    prompt = f"""Sei un giudice esperto nella valutazione di sistemi
16    di Information Extraction.
17    Devi confrontare il testo estratto (Parsed Text) con il testo di
18    riferimento (Gold Standard).
19    Valuta l'accuratezza semantica del testo estratto assegnando un
20    punteggio da 1 a 5, dove:
21    1 = Completamente errato o mancante
22    5 = Perfettamente coerente e accurato
23
24    Parsed Text:
25    {safe_parsed}
26
27    Gold Standard:
28    {safe_gold}
29
30    Rispondi solo ed esclusivamente con un JSON nel seguente formato,
31    senza aggiungere testo prima o dopo:
32    {{
33        "judge_score": <inserisci il voto qui>,
34        "judge_feedback": "<inserisci la motivazione qui>"
35    }}
36
37    Attenzione: Alcuni testi possono contenere tabelle formattate in
38    Markdown con il simbolo |. Valuta il loro contenuto semantico ma
39    non concentrarti solo su queste e non tentare in alcun modo di
40    correggere, completare o formattare queste tabelle. Concentrati
41    solo sull'estrazione del JSON.
42    """

```

Nella modalità *reference-free* invece, viene detto all'LLM di comparare il testo pulito direttamente con l'HTML sorgente della pagina web, preventivamente ripulito dai tag non informativi grazie al modulo *clean_html.py*. In entrambi i casi, è stata inserita una direttiva specifica per la gestione delle tabelle, in modo tale che l'LLM riesca a valutarne il contenuto semantico senza cercare di completarle o formattarle, evitando errori nell'output, come illustrato nel Listato 4.2.

Listing 4.2. Prompt per la modalità di valutazione reference-free

```

1 @app.post("/evaluate_judge_no_gs", response_model=
2     EvaluateJudgeResponse)
3
4 async def evaluate_judge_no_gs(data: EvaluateNoGsRequest):
5
6     char_limit = 1000
7
8     safe_parsed = data.parsed_text[:char_limit]
9     if len(data.parsed_text) > char_limit:
10         safe_parsed += "\n[...truncated text for time...]"
11
12     safe_html = clean_html(data.html_text, char_limit)

```

```

12  # 3. Prompt no gs for Llama 3.2
13  prompt = f"""Sei un giudice esperto di Information Extraction.
14  Devi valutare la qualita di un'estrazione testuale da una pagina
    web, SENZA avere un testo di riferimento.
15  Ti verranno forniti:
16  1. L'HTML grezzo della pagina web.
17  2. Il testo estratto dal parser.
18
19  Valuta da 1 a 5 se il testo estratto cattura il contenuto
    informativo principale presente nell'HTML, ignorando eventuali
    elementi strutturali mancanti.
20  1 = Testo del tutto irrilevante o estrazione fallita.
21  5 = Estrazione eccellente e coerente con l'HTML.
22
23  HTML Sorgente:
24  {safe_html}
25
26  Parsed Text:
27  {safe_parsed}
28
29  Rispondi solo ed esclusivamente con un JSON nel seguente formato:
30  {{
31      "judge_score": <inserisci il voto qui>,
32      "judge_feedback": "<inserisci la motivazione qui>"
33  }}
34  Attenzione: Alcuni testi possono contenere tabelle formattate in
    Markdown con il simbolo |. Valuta il loro contenuto semantico ma
    non concentrarti solo su queste e non tentare in alcun modo di
    correggere, completare o formattare queste tabelle. Concentrati
    solo sull'estrazione del JSON.
35  """

```

4.4 Determinismo del modello e validazione strutturale

I *Large Language Models* sono per natura stocastici, le risposte generate non sono deterministiche, ma basate su distribuzioni di probabilità apprese dai dati di *training*, quindi a parità di input possono generare output differenti e andare incontro ad allucinazioni, comportandosi spesso come "pappagalli stocastici" che mettono insieme parole senza una reale comprensione del significato [11]. Per il ruolo di valutatore, però, è fondamentale la riproducibilità e il determinismo dei risultati; per cercare di avvicinarsi a questo obiettivo il parametro `temperature` è stato impostato a 0.0 nelle opzioni della richiesta API. Inoltre, per facilitare l'estrazione automatica del voto e della motivazione da parte del *backend*, è stata aggiunta la direttiva `format="json"`. Questa opzione obbliga il modello a restituire esclusivamente un oggetto JSON contenente i campi `judge_score` e `judge_feedback`, in modo da eliminare testo aggiuntivo non richiesto. È mostrato nel Listato 4.3.

Listing 4.3. Impostazione dei parametri del modello.

```

1  target_model = "llama3.2"
2  payload = {
3      "model": target_model,
4      "prompt": prompt,

```

```

5         "stream": False,
6         "format": "json",
7         "options": {
8             "temperature": 0.0
9         }
10    }

```

Nonostante queste costrizioni, per prevenire il più possibile gli errori dovuti alle allucinazioni dell'LLM, è stata anche implementata una logica di tolleranza ai *fallback*, come mostrato nel Listato 4.4. Il sistema cerca prima di decodificare il JSON e di convertire il punteggio, assicurandosi che rientri nell'intervallo compreso tra 1 e 5. Nel caso in cui il modello ignori la specifica di formattazione e restituisca testo libero, viene intercettata l'eccezione `JSONDecodeError` e restituito uno *score* pari a 0, registrando l'anomalia nel campo di *feedback* senza interrompere l'esecuzione dell'intera *pipeline* di valutazione.

Listing 4.4. Logica di validazione e *fallback* per le risposte dell'LLM.

```

1  try:
2      async with httpx.AsyncClient(timeout=1800.0) as client:
3          response = await client.post(OLLAMA_URL, json=payload)
4          response.raise_for_status()
5
6          result = response.json()
7          raw_response = result.get("response", "")
8
9          try:
10             judge_output = json.loads(raw_response)
11             raw_score = judge_output.get("judge_score", 1)
12             try:
13                 # Controlla se l'intervallo è valido
14                 safe_score = max(1, min(5, int(float(raw_score))))
15             except (ValueError, TypeError):
16                 safe_score = 1
17
18             return EvaluateJudgeResponse(
19                 model_name=result.get("model", target_model),
20                 judge_score=safe_score,
21                 judge_feedback=judge_output.get("judge_feedback", "No
22                 feedback generated")
23             )
24
25         except json.JSONDecodeError:
26             # Fallback se l'LLM non rispetta il formato richiesto
27             # ritorna errore e \textit{score} 0
28             return EvaluateJudgeResponse(
29                 model_name=result.get("model", target_model),
30                 judge_score=0,
31                 judge_feedback=f"FALLBACK: l'LLM non rispetta il
32                 formato JSON. Raw response: {raw_response[:100]}..."
33             )
34
35     except httpx.ReadTimeout:
36         raise HTTPException(status_code=504, detail="Timeout durante la
37         generazione")
38
39     except Exception as e:

```

```
36 |     raise HTTPException(status_code=500, detail=f"Errore interno: {  
    |         str(e)}")
```

Capitolo 5

Risultati ottenuti

Questo capitolo è dedicato alla presentazione, all'analisi e al confronto critico dei risultati ottenuti durante la fase di test del sistema. L'obiettivo è illustrare le performance dei *parser* sviluppati, confrontando l'efficacia delle metriche di validazione tradizionali e avanzate con il giudizio fornito dal paradigma innovativo *LLM-as-a-Judge*, per ognuno dei quattro domini assegnati.

5.1 Impostazione degli esperimenti

Per condurre una valutazione rigorosa del sistema, è stato costruito un *dataset* di test composto da 40 pagine web, distribuite equamente tra i domini assegnati (10 URL per ciascun dominio). La selezione degli indirizzi non è stata casuale, ma guidata dalla necessità di scegliere contenuti informativi adatti all'addestramento e all'interrogazione utente di un modello come Chat Minerva. Nello specifico:

- Per le fonti enciclopediche e informative (en.wikipedia.org e viaggi-usa.it), sono state selezionate pagine relative a conoscenze generali, storiche o concetti di base, ad esempio la pagina sull'*Artificial Intelligence*.
- Per i domini giornalistici (cbsnews.com e cnbc.com), sono stati scelti articoli di attualità, in modo da testare l'algoritmo su strutture ricche di metadati, autori e date di pubblicazione.

I *Gold standard* per queste 40 pagine sono stati creati manualmente. Il processo ha previsto l'estrazione del solo contenuto utile, eliminando rumore visivo, immagini (conservandone però le didascalie informative), banner pubblicitari e riferimenti incrociati. Il titolo e il corpo dell'articolo sono stati opportunamente separati nei rispettivi campi del JSON e salvati in modo permanente all'interno del database.

Sfide strutturali dei domini

L'implementazione dei *parser* ha richiesto un approccio specializzato per affrontare la diversità delle strutture HTML dei domini analizzati. Le criticità principali affrontate durante gli esperimenti includono:

- **Complessità strutturale (en.wikipedia.org):** Wikipedia presenta un'architettura del DOM estremamente complessa. È stato necessario sviluppare script JavaScript e complesse *regex* per rimuovere *infobox*, tabelle riassuntive, sezioni di voci correlate ed enormi quantità di note bibliografiche o tag redazionali (es. [edit], [citation needed]), salvaguardando però il testo continuo.
- **Contenuti dinamici (cbsnews.com e cnbc.com):** pur avendo un *layout* più lineare, le testate giornalistiche presentano frequenti aggiornamenti. Le pagine più recenti subiscono modifiche all'HTML per inserire aggiornamenti sulle vicende o *link* ad articoli correlati. Questo genera un disallineamento se il *parsing* viene effettuato in modalità *live* rispetto al *Gold standard* salvato localmente settimane prima. Per mitigare questo effetto, i test sono stati condotti utilizzando l'HTML storicizzato in locale, garantendo così coerenza temporale. Nonostante questo problema, l'estrazione su questi domini ha registrato i livelli di accuratezza più elevati.
- **Incoerenza intra-dominio (viaggi-usa.it):** non trattandosi di un portale di livello *enterprise*, questo dominio presentava una forte varianza strutturale tra una pagina e l'altra. Tag HTML che in un URL contenevano informazioni cruciali, in un'altra pagina venivano utilizzati per elementi di *design* e viceversa, richiedendo un lavoro di estrema precisione e bilanciamento sui selettori.

I risultati quantitativi e qualitativi esposti nelle sezioni successive sono stati calcolati aggregando le valutazioni dei 10 URL per ciascun dominio e calcolandone la media aritmetica. I giudizi di ogni singola pagina sono stati ottenuti confrontando il *Gold standard* con il *parsed text* per le metriche tradizionali, quelle avanzate e l'LLM in configurazione classica; per l'LLM in configurazione *Reference-free*, invece, il testo pulito è stato comparato direttamente con il proprio HTML grezzo. Di conseguenza, il sistema è in grado di elaborare un qualsiasi indirizzo richiesto appartenente ad uno dei quattro domini supportati, ma se questo non è presente nella base di dati, potrà ricevere esclusivamente la valutazione dal paradigma *LLM-as-a-Judge Reference-free*, poiché tutte le altre metriche richiedono obbligatoriamente la presenza del testo di riferimento nel database.

5.2 Analisi delle metriche tradizionali e avanzate

La prima fase della valutazione ha previsto il confronto diretto tra il testo estratto dai *parser* e il rispettivo *Gold standard*, utilizzando sia metriche basate sulla sovrapposizione esatta dei *token* (*Token-level-eval*), sia metriche più avanzate (*X-eval*), in grado di valutare la vicinanza semantica e strutturale. Come è mostrato dai dati riportati in Tabella 5.1, i risultati mostrano un livello di accuratezza estremamente elevato su tutti i fronti. In particolare, il dominio giornalistico *cbsnews.com* raggiunge un punteggio perfetto (1.0000) nelle metriche *token-level*, a dimostrazione di un *parsing* chirurgico del contenuto. Leggermente inferiore, anche se ampiamente sopra la soglia del 99%, è il punteggio del dominio *viaggi-usa.it*: questo calo fisiologico è giustificato dall'incoerenza strutturale intra-dominio discussa nella sezione

precedente, che in alcuni casi ha portato alla conservazione di frammenti testuali secondari per non andare ad eliminare contenuti informativi importanti.

Dominio	Metriche Token-level-eval			Metriche X-eval				
	Precision	Recall	F1-score	Jaccard S.	Bigram O.	Cosine S.	METEOR	BERTScore
en.wikipedia.org	0.9979	0.9989	0.9984	0.9969	0.9983	0.9998	0.9981	1.0000
www.cbsnews.com	1.0000	1.0000	1.0000	1.0000	1.0000	0.9999	1.0000	1.0000
www.cnbc.com	0.9995	0.9985	0.9990	0.9981	0.9992	0.9986	0.9942	0.9935
www.viaggi-usa.it	0.9919	0.9989	0.9953	0.9909	0.9976	0.9988	1.0000	0.9962

Tabella 5.1. Valutazione dell'accuratezza per dominio: confronto tra metriche tradizionali e avanzate.

Le metriche classiche, però, mostrano un limite intrinseco: penalizzano qualsiasi variazione sintattica o di formattazione. Proprio per questo, come spiegato precedentemente, è stata necessaria l'integrazione di metriche semantiche come METEOR e BERTScore, che hanno permesso di validare l'estrazione non solo a livello testuale, ma anche a livello di significato, dimostrando che anche in presenza di minime variazioni sintattiche o rumore residuo, il contenuto informativo originale rimane solido.

Analizzando più nel dettaglio il caso di `viaggi-usa.it`, si nota l'importanza di questo doppio livello di valutazione. Nonostante la *Jaccard Similarity* (0.9909) e la *Precision* (0.9919) registrino i valori più bassi dell'intero *dataset*, penalizzando l'algoritmo per aver estratto qualche parola in più rispetto al testo di riferimento, la metrica METEOR restituisce il punteggio massimo. Questo risultato è fondamentale: poiché METEOR non si basa solo sulla precisa corrispondenza testuale, ma anche sui sinonimi, assicura che il testo estratto sia fluido e che il significato della frase sia totalmente preservato.

Un'ulteriore conferma della validità dei *parser* arriva da `en.wikipedia.org`. Anche se la sfida per la rimozione strutturale di *infobox*, note e rimandi esterni era notevole, il BERTScore registra un valore pari a 1.0000 dimostrando che la logica dei concetti è stata mantenuta intatta.

Per quanto riguarda il dominio `www.cnbc.com`, si osserva un comportamento particolare: le metriche basate sui *token* sfiorano la perfezione, ma si registra una lievissima flessione nei punteggi semantici. Questa discrepanza è fisiologica e dipende dalla natura prevalentemente economica degli articoli, ricchi di *ticker* azionari, acronimi aziendali e valori percentuali. I modelli linguistici alla base delle metriche semantiche tendono ad essere sensibili alla formattazione delle entità numeriche, penalizzando semanticamente anche la minima variazione.

Complessivamente i risultati garantiscono l'eccellente qualità dell'infrastruttura sviluppata. Infatti, tutte le metriche si attestano su valori prossimi o superiori al 99%, dimostrando che riescono ad estrarre informazione pulita, pronta per essere fornita in input ad un Large Language Model. Questa fase della sperimentazione, però, ha anche confermato il principale collo di bottiglia operativo delle metriche classiche e avanzate: la totale dipendenza dal *Gold standard*. Questo limite rende di fatto impossibile scalare e automatizzare il processo di valutazione su migliaia di URL in tempo reale o su indirizzi mai esplorati in precedenza, per questo è

stata necessaria l'introduzione di un paradigma di giudizio totalmente autonomo, *LLM-as-a-Judge Reference-free*, i cui risultati sono descritti nella sezione successiva.

5.3 Valutazione LLM-as-a-Judge

Come evidenziato alla fine della sezione precedente, il contributo più innovativo del sistema risiede nell'integrazione del paradigma *LLM-as-a-Judge*. In questa fase, il modello linguistico locale (Llama 3.2, 3B) è stato interrogato in due modalità distinte: una classica, sempre basata sul confronto con il *Gold standard*, e una *Reference-free*, in cui il modello ha agito in totale autonomia comparando il testo estratto con il codice HTML grezzo.

La Tabella 5.2 illustra i punteggi medi assegnati dall'LLM per i quattro domini. I risultati ottenuti con l'uso del testo di riferimento confermano i dati delle metriche tradizionali, attestandosi su valori estremamente alti, tra il 4.4 e il 5.0 (su una scala da 1 a 5).

Dominio	LLM Score con GS	LLM Score <i>Reference-free</i>
en.wikipedia.org	4.4	4.0
www.cbsnews.com	4.5	4.2
www.cnn.com	4.6	4.1
www.viaggi-usa.it	5.0	4.4

Tabella 5.2. Giudizi medi assegnati dal modello Llama 3.2 nelle due configurazioni.

Per quanto riguarda il sistema *Reference free*, si nota una lieve flessione nei punteggi che, però, non rappresenta minimamente un fallimento del progetto, ma una caratteristica fisiologica di questa modalità. L'HTML grezzo, infatti, contiene ogni singolo elemento della pagina (autori, date di pubblicazione, intestazioni) che invece il *parser* elimina, poiché progettato con questo scopo. L'LLM, senza il testo guida, nota questa discrepanza e penalizza leggermente il punteggio. Oltre allo *score* numerico il paradigma fornisce importanti *feedback* qualitativi che permettono un confronto più dettagliato tra i valori, poiché il modello non si limita ad applicare una formula matematica, ma dimostra una reale capacità di comprensione della pagina. Un primo esempio di questa capacità di ragionamento è emerso durante il collaudo del dominio *cbsnews.com*, analizzando un articolo medico il modello ha assegnato un punteggio pieno nella modalità con il *Gold standard*:

"Score: 5/5. Il testo estratto è perfettamente coerente e accurato, con la stessa struttura e contenuto del testo di riferimento."

Interrogando lo stesso modello nella forma *Reference-free* il giudizio è diventato più severo, abbassandosi a 4/5 con la seguente motivazione:

"Score: 4/5. Il testo estratto cattura bene il contenuto informativo principale della pagina web, ma manca alcuni dettagli specifici come la citazione di Dr. Celine Gounder e l'elenco delle conclusioni dello studio. L'estrazione è coerente con l'HTML e rappresenta un buon lavoro."

L'LLM si accorge autonomamente che il *parser*, per pulire la pagina dal rumore, ha ommesso una citazione che era invece presente e ben visibile nel codice HTML sorgente. Un altro caso simile riguarda un articolo finanziario su www.cnbc.com, relativo all'andamento azionario di Nvidia. Nel paradigma classico, il modello conferma l'ottimo lavoro dell'algoritmo:

"Score: 5/5. Il testo estratto è perfettamente coerente e accurato con il Gold Standard, con poche piccole differenze di stile ma senza alcun errore semantico significativo."

Senza il *Gold standard*, invece, il modello avendo accesso all'intero DOM, inclusa l'intestazione dell'articolo, nota l'assenza di questi dati e abbassa il voto a 4/5, dando questa motivazione:

"Score: 4/5. Il testo estratto cattura bene il contenuto informativo principale della pagina web, come la notizia sulla tendenza positiva dei titoli di azione di Nvidia. Tuttavia, manca alcune informazioni come il nome del giornalista o l'origine della notizia."

Questa analisi garantisce comunque il successo dell'architettura progettata. L'LLM in modalità *Reference-free* agisce come un vero giudice umano: penalizza l'estrazione parziale e giustifica la sua scelta fornendo all'utente un'indicazione precisa su dove migliorare l'estrazione. Questo paradigma è, quindi, da considerarsi uno strumento ingegneristico fondamentale per valutare e scalare l'elaborazione dei dati per sistemi complessi come Chat Minerva, superando definitivamente la necessità di una lunga azione di scrittura manuale.

5.4 Limiti del sistema

Nonostante i test certifichino l'elevata precisione dei *parser*, l'infrastruttura sviluppata presenta alcune limitazioni. In ottica di un'applicazione su larga scala, è fondamentale analizzare i colli di bottiglia computazionali e architetturali emersi durante gli esperimenti.

Il primo limite importante riguarda il costo computazionale e le tempistiche. L'esecuzione in locale di un Large Language Model da 3 miliardi di parametri richiede notevoli risorse hardware. A differenza del calcolo delle metriche classiche, la cui esecuzione è quasi istantanea, la generazione del *feedback* qualitativo e dello *score* quantitativo da parte dell'LLM può richiedere molti minuti per singola pagina, in base alle caratteristiche del calcolatore usato. Poiché i test sono stati eseguiti su un elaboratore standard, è stato necessario configurare il *client* HTTP asincrono con un *timeout* estremamente elevato, pari a 1800 secondi, specifica che potrebbe non essere adatta per un'elaborazione in tempo reale di migliaia di URL su tutti i tipi di dispositivi.

Strettamente collegato a questo problema è il limite della *Context Window* del modello. Per evitare l'esaurimento della memoria RAM e il conseguente arresto del sistema, è stato necessario il troncamento preventivo dell'input a 1000 caratteri. Sebbene questa limitazione sia facilmente eliminabile, se si volesse testare il sistema su potenti calcolatori, e i prompt siano stati ingegnerizzati inserendo l'avviso

esplicito [...truncated text for time...], questa scelta comporta comunque una valutazione parziale della pagina, sui dispositivi comuni. Di conseguenza, gli esperimenti fatti hanno giudicato l'accuratezza del *parser* basandosi esclusivamente sull'estrazione della prima porzione del documento.

Infine, un'ulteriore criticità è intrinseca nella natura stocastica del modello. Nonostante i parametri di generazione siano stati resi il più deterministici possibile (`temperature = 0.0`) e sia stata imposta la regola `format="json"`, durante la fase di test si sono verificati episodi isolati di allucinazioni in cui Llama 3.2 ha ignorato la direttiva del formato, restituendo la stringa JSON preceduta da testo discorsivo, ad esempio: *"Ecco la valutazione richiesta: {..."*. Questi casi limite hanno confermato la necessità della logica di tolleranza ai guasti implementata nel *backend*. Infatti, il blocco `try-except` ha intercettato l'eccezione `JSONDecodeError`, assegnando uno *score* pari a 0 e notificando l'utente dell'anomalia tramite il *feedback*, evitando così il collasso dell'intero *server*.

Questi limiti evidenziano come sia necessaria un'impalcatura di *software engineering* estremamente robusta per il paradigma *LLM-as-a-Judge*, in modo da mitigare le instabilità intrinseche degli LLM.

Capitolo 6

Conclusioni e sviluppi futuri

L'obiettivo principale di questa tesi è stato progettare, sviluppare e validare un'infrastruttura ingegneristica per l'estrazione e la valutazione di contenuti web, superando i limiti di scalabilità dei tradizionali approcci basati sul confronto con testi di riferimento. Nel panorama tecnologico attuale, dove i Large Language Models richiedono enormi quantità di dati puliti per l'addestramento e l'alimentazione di sistemi *Retrieval-Augmented Generation*, la capacità di automatizzare il *parsing* rappresenta una necessità fondamentale. Infatti, è necessario che gli LLM, come è servito per il progetto Chat Minerva, abbiano dei solidi sistemi di estrazione che consentano ai modelli di trovare facilmente tutte le informazioni necessarie per rispondere, in modo preciso, alle interrogazioni da parte degli utenti, riducendo così al minimo il rischio di allucinazioni.

Nel corso dei capitoli precedenti è stata presentata l'implementazione di un'architettura completa, basata su un *backend* FastAPI, in grado di interrogare domini web di tutti i tipi (enciclopedici, giornalistici e informativi) ed estrarre il contenuto utile pulendo tutte le informazioni di rumore tramite *parser* HTML appositi. Per confermare la robustezza di questi script, è stata condotta un'analisi comparativa su tre livelli di complessità crescente.

Inizialmente, tramite l'utilizzo di metriche *token-level* (F1-score, Precision e Recall) e di metriche avanzate (Jaccard similarity, Bigram overlap, Cosine similarity, METEOR e BERTScore), è stata certificata l'eccellente qualità dell'estrazione testuale, registrando valori prossimi o superiori al 99% su tutti i domini testati. Tuttavia, questa fase ha anche confermato il limite strutturale di questi metodi: l'assoluta dipendenza da un'onerosa scrittura manuale.

Per superare questo limite, il lavoro si è concentrato sull'integrazione del modello Llama 3.2 come valutatore autonomo. Il collaudo del paradigma *LLM-as-a-Judge*, in particolare nella sua configurazione *Reference-free*, ha dimostrato che è possibile delegare all'Intelligenza Artificiale non solo il calcolo di uno *score* di accuratezza analizzando direttamente l'HTML grezzo, ma anche la generazione di un *feedback* preciso che permette di capire i possibili problemi dei *parser*. Questa architettura rende la validazione dei dati finalmente scalabile e indipendente dai *dataset* manuali.

Nonostante gli eccellenti risultati ottenuti, l'infrastruttura sviluppata può essere ancora migliorata, attraverso diversi sviluppi futuri volti a risolverne le attuali limitazioni. In primo luogo, per superare il collo di bottiglia dei lunghi tempi di attesa

con l'attuale *timeout* di 1800 secondi, il sistema potrebbe essere evoluto integrando un *message broker* per la gestione asincrona a code. In questo modo si potrebbero accodare migliaia di URL da validare in *background*, delegando l'elaborazione a *worker* distribuiti e rendendo l'architettura realmente scalabile.

In secondo luogo, l'esecuzione del modello su potenti calcolatori dotati di GPU dedicate e di enormi quantità di RAM, consentirebbe di rimuovere l'attuale troncamento preventivo dell'input a 1000 caratteri. Questa espansione della *Context Window* permetterebbe all'LLM di analizzare alberi DOM di qualsiasi profondità, garantendo una valutazione completa su tutta la pagina, senza tralasciare alcun dettaglio.

Infine, un'ulteriore evoluzione del progetto potrebbe prevedere la sostituzione del modello testuale con un Vision-Language Model. Fornendo al valutatore uno *screenshot* visivo della pagina renderizzata nel browser, oltre al codice sorgente, il sistema acquisirebbe una reale comprensione del *layout* grafico, permettendo di verificare se il processo di estrazione ha alterato la struttura logica voluta dagli autori originali.

Bibliografia

- [1] Patrick Lewis et al. «Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks». In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474.
- [2] Amazon Web Services (AWS). *Cos'è la Retrieval-Augmented Generation (RAG)?* URL: <https://aws.amazon.com/it/what-is/retrieval-augmented-generation/> (visitato il giorno 31/08/2026).
- [3] Docker Inc. *Docker: Accelerated Container Application Development*. <https://www.docker.com/>. Ultimo accesso: 31 agosto 2026.
- [4] Sebastián Ramírez. *FastAPI*. <https://fastapi.tiangolo.com/>. Ultimo accesso: 1 settembre 2026.
- [5] Pydantic Services Inc. *Pydantic: Data validation using Python type hints*. <https://pydantic.dev/docs/validation/latest/get-started/>. Ultimo accesso: 1 settembre 2026.
- [6] Encode OSS. *HTTPX: A next-generation HTTP client for Python*. <https://www.python-httpx.org/>. Ultimo accesso: 1 settembre 2026.
- [7] Crawl4AI Contributors. *Crawl4AI: Open-source LLM Friendly Web Crawler & Scraper*. <https://crawl4ai.com/>. Ultimo accesso: 2 settembre 2026.
- [8] Pallets. *Jinja Documentation (Stable)*. <https://jinja.palletsprojects.com/en/stable/>. Ultimo accesso: 2 settembre 2026.
- [9] MariaDB Foundation. *MariaDB - The open source relational database*. <https://mariadb.org/>. Ultimo accesso: 5 settembre 2026.
- [10] Ollama. *Ollama: Get up and running with large language models*. <https://ollama.com/>. Ultimo accesso: 6 settembre 2026.
- [11] Emily M. Bender et al. «On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?» In: *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*. FAccT '21. 2021, pp. 610–623.